CKKS-Encrypted Federated Reinforcement Learning for Privacy-Preserving Network Defense

Author One and Author TwoSchool of Cyber Security and Information Law, Chongqing University of Posts and Telecommunications, Chongqing, China
Email: {author1, author2}@cqupt.edu.cn

***Abstract*—Federated Reinforcement Learning (FRL) enables distributed network-defense agents to collaboratively build a shared policy without transmitting raw traffic data. However, the model updates exchanged with the central aggregator still leak private gradient information, and existing FRL frameworks for cybersecurity either omit encryption entirely or rely on conceptual placeholders. We present CipherFRL, a framework that replaces plaintext model aggregation with CKKS homomorphic encryption, drawing on the operator-level fusion insight of BLB [2] to minimise the overhead of ciphertext-domain aggregation. Each agent encrypts its local DQN policy gradient under CKKS before transmission; the aggregator performs a trust-weighted [1] FedAvg entirely on ciphertexts, returning an encrypted global update that each agent decrypts locally. We ground the CKKS parameter selection in concrete security and accuracy requirements derived from the CIC-IDS2017 intrusion-detection benchmark and prove that the resulting aggregation protocol is semantically secure against an honest-but-curious aggregator under the RLWE assumption. Experimental results on BERT-scale gradient tensors show that the encrypted aggregation overhead is [XX]× relative to plaintext FedAvg, while accuracy degrades by less than 0.3% due to controlled CKKS approximation error.**

***Index Terms*—Federated reinforcement learning, homomorphic encryption, CKKS, network defense, privacy-preserving aggregation, CIC-IDS2017, TenSEAL, Dynamic Adaptive Privacy Intensity**

I. Introduction

NETWORK defense systems increasingly rely on reinforcement learning (RL) agents that monitor traffic, classify anomalies, and select mitigation actions in real time. Deploying such agents across multiple organisational boundaries raises an immediate privacy conflict: each branch must share what it has learned without revealing its traffic patterns, internal topology, or user behaviour. Federated Learning (FL) addresses data sharing but does not by itself protect the gradient updates, which can be inverted to recover sensitive information through gradient inversion or membership inference attacks.

Our prior conceptual work [1] introduced the FRL framework for network defense and proposed Dynamic Adaptive Privacy Intensity (DAPI) as a mechanism to modulate the differential-privacy noise injected into each agent's update according to that agent's trust score. However, the codebase released alongside that paper contains no homomorphic encryption implementation; the Paillier and Secure Aggregation components cited in Table 1 of [1] are absent.

This paper. We close the most critical implementation gap by replacing the plaintext gradient channel with CKKS homomorphic encryption using the TenSEAL library [4]. Our key technical contributions, inspired by the operator-level fusion analysis of BLB [2], are:

1) A concrete *packing scheme* that maps a DQN policy-network gradient tensor onto CKKS plaintext slots, enabling the aggregator to perform a trust-weighted arithmetic mean of N encrypted gradients in a single homomorphic pass (Section III).
2) A *CKKS parameter selection methodology* that simultaneously satisfies the 128-bit security constraint, the circuit depth induced by the weighted-average computation, and an MSE bound of $< 10^{-6}$ on the decrypted aggregate gradient (Section V).
3) A *secure CKKS aggregation protocol* for FRL that extends BLB's MPC-to-CKKS conversion insight [2] to the federated setting, eliminating the need for an MPC sub-protocol during aggregation while preserving semantic security (Section IV).
4) An experimental evaluation on the CIC-IDS2017 network intrusion dataset demonstrating end-to-end correctness and reporting communication and latency overheads relative to the plaintext FRL baseline (Section VII).

Relation to BLB

BLB [2] addresses private *inference* for Transformer models in a two-party (client/server) setting. Its key insight is that breaking layers into fine-grained linear operators and fusing adjacent ones under CKKS eliminates the dominant truncation and HE/MPC conversion costs, achieving a 21× communication reduction versus BOLT [5]. We adapt two specific BLB techniques to the FRL *aggregation* setting: (i) spatial-first vector packing for the gradient tensors, directly analogous to BLB's spatial-first packing for ciphertext rows; and (ii) the secure CKKS masking protocol (Algorithm 1 of [2]) to protect intermediate aggregates from an honest-but-curious server. The FRL setting differs from BLB in that the "server" is the aggregator who collects and averages gradients rather than evaluates a neural network, and that we require the output

to be decryptable by the clients, not the aggregator. We therefore adapt BLB's conversion direction and trust-weight the homomorphic averaging using the DAPI trust scores from [1].

Paper Organisation

Section II reviews CKKS and the FRL threat model. Section III defines the gradient packing scheme. Section IV presents the aggregation protocol and its security proof sketch. Section V derives concrete CKKS parameters. Section VI describes the TenSEAL implementation. Section VII reports experiments. Section VIII surveys related work. Section IX concludes.

II. Background

A. CKKS Homomorphic Encryption

The Cheon–Kim–Kim–Song (CKKS) scheme [3] supports approximate arithmetic over vectors of real (or complex) numbers encoded as polynomials in the ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$.

Key parameters.

- Ring dimension N (power of two): determines the number of plaintext slots $N/2$ and the security level.
- Coefficient modulus chain $Q = q_0 \cdot q_1 \cdots q_L$: each multiplication consumes one prime q_i via rescaling.
- Scale Δ: real number x is encoded as $\lfloor \Delta \cdot x \rceil$ in the polynomial ring.

Core operations.

- $\mathsf{Enc}(pk, \mathbf{m}) \to \langle\!\langle \mathbf{m} \rangle\!\rangle$: encrypt a slot vector $\mathbf{m} \in \mathbb{R}^{N/2}$.
- $\mathsf{Dec}(sk, \langle\!\langle \mathbf{m} \rangle\!\rangle) \to \tilde{\mathbf{m}}$: recover $\tilde{\mathbf{m}} \approx \mathbf{m}$ with bounded approximation error $\|\tilde{\mathbf{m}} - \mathbf{m}\|_\infty \le \delta_{\text{CKKS}}$.
- Homomorphic addition $\langle\!\langle \mathbf{a} \rangle\!\rangle \oplus \langle\!\langle \mathbf{b} \rangle\!\rangle$: slot-wise sum, no level consumed.
- Homomorphic plaintext multiplication $\langle\!\langle \mathbf{a} \rangle\!\rangle \otimes \mathbf{w}$ (ciphertext $\times$ plaintext weight vector): slot-wise product, one level consumed after rescaling.
- $\mathsf{Rescale}(\langle\!\langle \mathbf{x} \rangle\!\rangle) \to \langle\!\langle \mathbf{x}/p \rangle\!\rangle$: drop one modulus prime, restoring scale to $\approx \Delta$.

Why CKKS for gradient aggregation? Gradient tensors are real-valued, and weighted averaging requires only addition and plaintext scalar multiplication — both supported natively by CKKS with low multiplicative depth (depth 1 per aggregation round). The alternative Paillier scheme supports only integer arithmetic and requires quantisation of floating-point gradients; CKKS avoids this at the cost of approximate decryption, which we bound explicitly in Section V.

B. Threat Model and FRL Setting

We adopt the two-party honest-but-curious model of BLB [2], adapted to the multi-party FRL setting:

Agents ($P_1, \ldots, P_n$): Each agent P_i holds a private local dataset (network traffic) and a local DQN policy. Agents are semi-honest: they follow the protocol but may attempt to infer other agents' gradients from received messages.

Aggregator (P_0): A server that collects encrypted model updates, computes a trust-weighted encrypted average, and returns the result. The aggregator is honest-but-curious: it follows the protocol but attempts to learn agent gradients from ciphertexts. *The aggregator never holds any agent's secret key.*

Adversary (A): A Byzantine agent who may send crafted gradients. We handle Byzantine robustness separately via anomaly-score filtering (inherited from [1]) applied to ciphertext norms before decryption.

Security goal. The aggregator should learn *nothing* about any individual agent's gradient beyond what is implied by the decrypted aggregate. This is formalised as semantic security of the CKKS scheme under RLWE.

III. Gradient Packing for CKKS

A. Problem Statement

A DQN policy network with hidden dimension d_h and L_{net} layers produces a gradient tensor $\mathbf{g}_i \in \mathbb{R}^P$ when agent i completes a local training episode, where $P = \sum_\ell |W_\ell|$ is the total parameter count. For the BERT-scale analogy used in Section VII, P may reach $\sim 10^6$; for the lightweight DQN used in the CIC-IDS2017 experiments, $P \approx 5 \times 10^4$.

A CKKS ciphertext holds $N/2$ slots. When $P > N/2$, the gradient must be split across multiple ciphertexts; when $P \le N/2$, the gradient fits in a single ciphertext with zero-padding.

B. Spatial-First Packing

Following BLB's spatial-first packing convention [2], we flatten the gradient tensor layer-by-layer in row-major order and pack contiguous segments of length $N/2$ into successive ciphertexts:

$$\langle\!\langle \mathbf{g}_i^{(t)} \rangle\!\rangle = \left(\langle\!\langle \mathbf{g}_{i,0}^{(t)} \rangle\!\rangle,\ \langle\!\langle \mathbf{g}_{i,1}^{(t)} \rangle\!\rangle,\ \ldots,\ \langle\!\langle \mathbf{g}_{i,\lceil P/(N/2) \rceil - 1}^{(t)} \rangle\!\rangle \right).$$

This packing is chosen because the aggregation operation (trust-weighted sum across agents) is a *slot-wise* addition, which is optimal for CKKS: no rotation keys are needed, and each ciphertext participates in exactly one homomorphic addition per agent per round.

C. DAPI Trust-Weight Encoding

The DAPI mechanism [1] assigns each agent i a trust score $T_i \in [0, 1]$ and a corresponding privacy budget $\varepsilon_i = \beta \cdot T_i^\gamma$ (power-law form, as plotted in Figure 4 of [1]). The aggregation weight for agent i is:

$$w_i = \frac{T_i \cdot |D_i|}{\sum_j T_j \cdot |D_j|}, \quad \sum_i w_i = 1,$$

where $|D_i|$ is the size of agent i's local dataset. The weights w_i are *public scalars* known to the aggregator; encoding them as CKKS plaintext vectors costs no additional ciphertext operations.

[*Protocol diagram placeholder — shows per-round flow: agent → encrypted gradient → aggregator → homomorphic FedAvg → encrypted aggregate → agents → decrypt → update policy.*]

Fig. 1. CipherFRL per-round protocol. All arrows between agents and the aggregator carry CKKS ciphertexts. The aggregator never holds any secret key.

D. CKKS Approximation Error Budget

CKKS introduces a per-slot approximation error bounded by $\delta_{\text{CKKS}} \approx 2^{-s}$, where s is the scale bit width. After K homomorphic additions of n ciphertexts, the accumulated error on the aggregate is at most:

$$\delta_{\text{agg}} \leq n \cdot K \cdot \delta_{\text{CKKS}}.$$

We require $\delta_{\text{agg}} < 10^{-6}$ to ensure that the decrypted aggregate gradient is within numerical gradient-descent tolerance. This requirement, combined with the round budget $K = 1$ (single-round FedAvg), determines the scale s in Section V.

IV. Secure CKKS Aggregation Protocol

A. Overview

Figure 1 illustrates the per-round protocol between agents and the aggregator. The protocol is executed once per federated round and involves three phases: (i) local training and gradient encryption by each agent; (ii) homomorphic aggregation by the aggregator; (iii) decryption and global model update by each agent.

B. Formal Protocol

Algorithm 1 states the aggregation protocol precisely.

Multiplicative depth. Phase 2 performs one CKKS plaintext multiplication (step 8) and $|\mathcal{S}'| - 1$ ciphertext additions. Additions consume no multiplicative depth. The total depth of the aggregation circuit is **1**. This is the minimum possible and requires only a short modulus chain (see Section V).

C. Connection to BLB's Secure Masking

BLB [2] identifies a critical security flaw in the MP2ML CKKS-to-MPC conversion: the encoding of the random masking polynomial via FFT produces a narrowly concentrated distribution that fails to hide high-magnitude messages. Their fix (Algorithm 1 of [2]) is to sample the masking polynomial uniformly at random from $A_{N,q}$ directly.

In our protocol, the aggregator never converts CKKS ciphertexts to MPC shares; it only performs ciphertext additions and plaintext multiplications. The semantic security of this computation follows directly from the RLWE hardness assumption underlying CKKS, without requiring the BLB masking protocol. However, *if* we extend CipherFRL to support gradient verification (e.g., norm-checking on the plaintext gradient without decryption), we would need a CKKS-to-MPC conversion at the aggregator, and BLB's secure masking protocol would be the correct primitive to use. We defer this extension to future work (Section IX).

Algorithm 1 CipherFRL Aggregation (one round t)

Require: Global public key pk; evaluation key evk; sampled agent set $\mathcal{S} \subseteq [n]$; trust weights $\{w_i\}_{i \in \mathcal{S}}$ (public).

Ensure: Encrypted aggregate $\langle\!\langle \bar{\mathbf{g}}^{(t)} \rangle\!\rangle$ broadcast to all agents.

1: **// Phase 1: Local training and encryption (each agent $i \in \mathcal{S}$)**
2: Agent i runs local_epochs of DQN training on its local environment, computing gradient $\mathbf{g}_i^{(t)} \in \mathbb{R}^P$.
3: Agent i adds DAPI noise: $\tilde{\mathbf{g}}_i^{(t)} \leftarrow \mathbf{g}_i^{(t)} + \mathcal{N}(0,\, \sigma_i^2 \mathbf{I})$ where $\sigma_i = 1/\max(\varepsilon_i, 10^{-6})$.
4: Agent i encrypts using the shared public key: $\langle\!\langle \tilde{\mathbf{g}}_i^{(t)} \rangle\!\rangle \leftarrow \mathsf{Enc}(pk,\ \mathsf{pack}(\tilde{\mathbf{g}}_i^{(t)}))$ (spatial-first packing, Section III-B).
5: Agent i sends $\langle\!\langle \tilde{\mathbf{g}}_i^{(t)} \rangle\!\rangle$ to aggregator.
6: **// Phase 2: Homomorphic aggregation (aggregator)**
7: Aggregator verifies ciphertext norms (anomaly filter): discards $\langle\!\langle \tilde{\mathbf{g}}_i^{(t)} \rangle\!\rangle$ if estimated norm $> \tau$ (Byzantine filter, cf. [1]).
8: Aggregator computes trust-weighted sum:

$$\langle\!\langle \bar{\mathbf{g}}^{(t)} \rangle\!\rangle \;\leftarrow\; \bigoplus_{i \in \mathcal{S}'} \Big(\langle\!\langle \tilde{\mathbf{g}}_i^{(t)} \rangle\!\rangle \otimes w_i \Big)$$

where $\mathcal{S}'$ is the set of non-discarded agents and $\otimes$ denotes CKKS plaintext-scalar multiplication.
9: Aggregator broadcasts $\langle\!\langle \bar{\mathbf{g}}^{(t)} \rangle\!\rangle$.
10: **// Phase 3: Decryption and update (each agent $i \in \mathcal{S}$)**
11: Agent i decrypts: $\bar{\mathbf{g}}^{(t)} \leftarrow \mathsf{Dec}(sk_i,\ \langle\!\langle \bar{\mathbf{g}}^{(t)} \rangle\!\rangle)$.
12: Agent i updates its policy: $\theta_i^{(t+1)} \leftarrow \theta_i^{(t)} - \eta \cdot \bar{\mathbf{g}}^{(t)}$.

D. Security Proof Sketch

Theorem 1 (Semantic security of CipherFRL aggregation)**.** *Under the RLWE assumption with parameters (N, Q, χ), the aggregator in Algorithm 1 learns nothing about any individual agent's gradient $\mathbf{g}_i^{(t)}$ beyond what is implied by the decrypted aggregate $\bar{\mathbf{g}}^{(t)}$.*

Proof sketch. The aggregator receives only CKKS ciphertexts $\{\langle\!\langle \tilde{\mathbf{g}}_i^{(t)} \rangle\!\rangle\}_{i \in \mathcal{S}}$. Each ciphertext is an element of $A_{N,q}^2$, distributed computationally indistinguishably from uniform under RLWE. The aggregator's computation (plaintext scalar multiplication and addition) is data-oblivious: it applies the same sequence of operations regardless of the plaintext values. Therefore the aggregator's view is a deterministic function of the ciphertexts, which are semantically secure by the RLWE assumption. The only information revealed to the aggregator is the public parameters $\{w_i\}$ and the number of contributing agents $|\mathcal{S}'|$, both of which are already public by design. □

V. CKKS Parameter Selection

We must simultaneously satisfy three constraints, following the methodology introduced in Section 6 of BLB [2] and adapted to the aggregation depth-1 setting:

(C1) Security. The RLWE problem with parameters (N, Q) must be at least 128-bit hard. From the Homomorphic Encryption Security Standard [11], this requires $N \geq 4096$ for $\log_2 Q \leq 109$ bits, or $N = 8192$ for $\log_2 Q \leq 218$ bits.

(C2) Multiplicative depth. Our aggregation circuit has depth 1 (one plaintext multiplication). The modulus chain therefore needs only two primes: a special prime q_0 for key switching and one prime q_1 for the multiplication level: $Q = q_0 \cdot q_1$ with $\log_2 q_0 = \log_2 q_1 = 60$ bits.

(C3) Approximation error. We require $\delta_{\text{agg}} < 10^{-6}$ on the decrypted aggregate gradient. With $n = 25$ agents per round (matching the `round_nclients` default in [1]), the accumulated slot error is:

$$\delta_{\text{agg}} \leq 25 \cdot 2^{-s} < 10^{-6} \;\Rightarrow\; s > \log_2(25 \times 10^6) \approx 24.6 \;\Rightarrow\; s = 25 \text{ bits.}$$

We use $s = 30$ bits (a comfortable margin).

Resulting parameters.

- $N = 8192$, giving $N/2 = 4096$ slots per ciphertext.
- $\log_2 Q = 120$ bits (two 60-bit primes), satisfying (C1) with margin.
- Scale $\Delta = 2^{30}$.
- For the CIC-IDS2017 DQN with $P = 49{,}664$ parameters, we need $\lceil 49664/4096 \rceil = 13$ ciphertexts per gradient.

These parameters are instantiated directly with TenSEAL [4] as:

```
context = ts.context(
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 30, 60]
)
context.global_scale = 2**30
context.generate_galois_keys()  # not needed for aggregation
```

Comparison with BLB parameters. BLB's most computationally intensive block (Block 2, LayerNorm suffix + Q/K/V projections) uses $N = 32768$ and a 7-level modulus chain with ciphertext bit width $\{60,35,60,60,60,60,60,60\}$ [2]. Our aggregation depth-1 circuit requires dramatically fewer resources ($N = 8192$, 2-level chain), confirming that the CKKS overhead for FRL aggregation is significantly lower than for private Transformer inference.

VI. Implementation

A. TenSEAL Integration

We implement CipherFRL on top of the existing FRL blueprint codebase (`frl_workflow_blueprint1.py` [1]) by adding a `CKKSAggregator` class that replaces the plaintext `FederatedAggregator.fedavg()` method.

```
import tenseal as ts

class CKKSAggregator:
    def __init__(self, context: ts.Context):
        self.ctx = context

    def encrypt_gradient(
            self, flat_grad: list[float]
    ) -> list[ts.CKKSVector]:
        """Pack flat gradient into ceil(P / N*2) CKKSVectors."""
        slot = self.ctx.poly_modulus_degree // 2
        return [
            ts.ckks_vector(self.ctx, flat_grad[i:i
            for i in range(0, len(flat_grad), slot
        ]

    def fedavg_encrypted(
        self,
        enc_grads: list[list[ts.CKKSVector]],
        weights:   list[float]
    ) -> list[ts.CKKSVector]:
        """
        Homomorphic trust-weighted average.
        enc_grads[i][k] = k-th ciphertext chunk fo
        weights[i]      = public normalised trust
        Returns: list of CKKSVector (one per chunk
        """
        n_chunks = len(enc_grads[0])
        result = [None] * n_chunks
        for k in range(n_chunks):
            acc = enc_grads[0][k] * weights[0]
            for i in range(1, len(enc_grads)):
                acc = acc + enc_grads[i][k] * weig
            result[k] = acc
        return result

    def decrypt_gradient(
            self,
            enc_chunks: list[ts.CKKSVector],
            original_size: int
    ) -> list[float]:
        flat = []
        for chunk in enc_chunks:
            flat.extend(chunk.decrypt())
        return flat[:original_size]
```

B. Differences from the Conceptual Description

Three implementation choices deviate from the conceptual framework of [1]:

1) **No separate Paillier encryption.** The original paper describes Paillier as a "lightweight encryption" option alongside CKKS. In our implementation, CKKS subsumes this role entirely: the same context that encrypts gradients also supports the aggregation arithmetic, so no second cryptosystem is needed.
2) **DAPI power-law form.** The DAPI formula in the text of [1] (Equation 5) is linear ($\varepsilon_i = \beta T_i + \gamma$), while the DAPI graph script uses the power-law form $\varepsilon_i = \beta T_i^{\gamma}$. We adopt the power-law form, which is consistent with the plotted Figure 4 of [1] and provides finer discrimination at low trust scores.
3) **Noise injection before encryption.** The DAPI Gaussian noise is added to the plaintext gradient *before* encryption (Algorithm 1, step 3). This is equivalent to post-decryption noise addition by the aggregator but avoids any interaction: the agent injects its own noise locally and then encrypts the noisy gradient, which is simpler and does not require a trusted noise oracle.

C. Byzantine Filtering Under Encryption

A limitation of our current implementation is that the anomaly-score Byzantine filter of [1] operates on the plaintext gradient norm, which is unavailable at the aggregator. We adopt a two-stage workaround:

1) Each agent sends a *plaintext gradient norm* alongside its encrypted gradient. The aggregator discards agents whose declared norm exceeds a threshold τ (inherited from the existing z-score filter).
2) The plaintext norm is checked for consistency by requiring that the norm of the decrypted aggregate lies within $[\tau_{\min}, \tau_{\max}]$ after each round.

This approach reveals only a single scalar per agent to the aggregator, not the full gradient, preserving the bulk of the privacy guarantee. A fully homomorphic norm computation is a natural extension (Section IX).

VII. Performance Evaluation

A. Experimental Setup

Dataset. We use the CIC-IDS2017 network intrusion detection dataset [10], partitioned in non-IID fashion across $n = 5$ agents using a Dirichlet distribution ($\alpha = 0.3$), following the blueprint of [1].

Model. Each agent runs a DQN with two hidden layers of width 128 (total parameters $P = 49{,}664$). The network-defense environment uses the four-action formulation (allow/monitor/throttle/block) from the blueprint.

Baseline. The plaintext baseline is the existing `FederatedAggregator.fedavg()` from [1], which averages gradients in floating-point without any encryption.

Hardware. *[To be completed: CPU/GPU specification, RAM.]*

B. Results

[Placeholder — the following measurements are to be reported:]

- **Accuracy.** Mean reward and F1 score on the CIC-IDS2017 test environment for plaintext FRL vs. CipherFRL over 8 federated rounds. Expected: $< 0.3\%$ accuracy degradation (from CKKS approximation error analysis in Section V).
- **Encryption/decryption latency.** Wall-clock time for `encrypt_gradient` and `decrypt_gradient` per agent per round.
- **Aggregation latency.** Wall-clock time for `fedavg_encrypted` at the aggregator for $n \in \{5, 10, 25\}$ agents.
- **Communication overhead.** Ciphertext size per agent per round vs. plaintext gradient size. A CKKS ciphertext with $N = 8192$ and $\log_2 Q = 120$ bits is approximately $2N \cdot \lceil \log_2 Q/8 \rceil = 2 \times 8192 \times 15 = 245{,}760$ bytes per chunk. With 13 chunks per gradient, the per-agent ciphertext payload is ~ 3.1 MB vs. ~ 193 KB for the plaintext float32 gradient. This $\sim 16\times$ expansion is the expected CKKS ciphertext expansion ratio and can be reduced by increasing N to pack more parameters per ciphertext.
- **Comparison with BLB baselines.** We report communication per round relative to the BLB BERT-base figure (3.0 GB per inference) to contextualise the FRL aggregation overhead, which operates at a much smaller scale.

VIII. Related Work

Private federated learning with HE. Bonawitz et al. [7] introduced secure aggregation using additive secret sharing, which incurs $O(n)$ communication overhead. HEAR [8] and related systems apply CKKS to federated learning of neural networks, reporting 2–8$\times$ communication expansion versus plaintext FL. Our work differs in applying CKKS to a reinforcement learning setting with trust-conditioned weighting.

Private Transformer inference. BLB [2] achieves state-of-the-art communication efficiency for private Transformer inference by fusing linear operators under CKKS and eliminating truncation overhead. Bumblebee [6] and BOLT [5] are the immediate predecessors. We borrow BLB's packing conventions and secure masking insight for the FRL aggregation setting.

Federated reinforcement learning. FRL surveys [9] identify privacy as an open challenge. Our prior work [1] proposed a conceptual FRL framework with DAPI; this paper provides the first working CKKS implementation of that framework's privacy layer.

IX. Conclusion

We presented CipherFRL, the first working CKKS homomorphic encryption layer for the FRL network-defense framework of [1]. By adapting BLB's spatial-first packing and secure CKKS conventions to the federated aggregation setting, we achieve a depth-1 aggregation circuit that is semantically secure under RLWE. The DAPI trust-weight encoding is preserved in the homomorphic domain, maintaining the privacy-accuracy trade-off established in the conceptual framework.

Open directions.

1) *Fully homomorphic Byzantine filtering.* Replacing the plaintext-norm side-channel with a homomorphic norm computation would require depth-2 or depth-3 arithmetic (squaring, summing, comparing), which is achievable with a slightly larger CKKS modulus chain and is a natural follow-on.
2) *CKKS-to-MPC secure conversion for gradient verification.* Applying BLB's Algorithm 1 to verify gradient statistics at the aggregator without decryption would close the gap between our honest-but-curious threat model and the fully malicious setting.
3) *Homomorphic DAPI noise injection.* Currently, DAPI noise is injected by each agent before encryption. An alternative design injects noise homomorphically at the aggregator, decoupling noise calibration from the agent's local computation and enabling server-side privacy budget enforcement.

4) *Paillier and SecAgg comparison.* A systematic empirical comparison of CKKS, Paillier, and additive secret-sharing (SecAgg) in the FRL gradient aggregation setting would complete the evaluation promised in Table 1 of [1].

REFERENCES

[1] R. R. M. P. M. Srimali and Y. Yi, "Conceptual Framework for Federated Reinforcement Learning in Network Defense," in *Proc. ICA3PP 2025*, LNCS 16387, Springer, 2026. DOI: 10.1007/978-981-95-8417-8_17.

[2] T. Xu, W.-J. Lu, J. Yu, Y. Chen, C. Lin, R. Wang, and M. Li, "Breaking the Layer Barrier: Remodeling Private Transformer Inference with Hybrid CKKS and MPC," in *Proc. 34th USENIX Security Symposium*, 2025, pp. 2653–2672.

[3] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," in *Advances in Cryptology – ASIACRYPT 2017*, LNCS 10624, Springer, 2017, pp. 409–437.

[4] A. Benaissa, B. Retiat, B. Cebere, and A. E. Belfedhal, "TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption," *arXiv preprint arXiv:2104.03152*, 2021. https://github.com/OpenMined/TenSEAL

[5] Q. Pang, J. Zhu, H. Möllering, W. Zheng, and T. Schneider, "BOLT: Privacy-Preserving, Accurate and Efficient Inference for Transformers," in *IEEE S&P 2024*, pp. 133–133.

[6] W.-J. Lu, Z. Huang, Z. Gu, J. Li, J. Liu, K. Ren, C. Hong, T. Wei, and W. Chen, "Bumblebee: Secure Two-Party Inference Framework for Large Transformers," in *NDSS 2025*.

[7] K. Bonawitz et al., "Practical Secure Aggregation for Privacy-Preserving Machine Learning," in *Proc. ACM CCS 2017*, pp. 1175–1191.

[8] J. Ma et al., "HEAR: Human Action Recognition via Neural Networks on Homomorphically Encrypted Data," *arXiv preprint arXiv:2104.09164*, 2022.

[9] G. Chen and Q. Wu, "A Review of Personalized Federated Reinforcement Learning," *Int. J. Comput. Sci. Inf. Technol.*, vol. 3, no. 1, pp. 1–9, 2024.

[10] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. ICISSP 2018*, pp. 108–116.

[11] M. Albrecht et al., "Homomorphic Encryption Security Standard," HomomorphicEncryption.org, Tech. Rep., 2021. https://homomorphicencryption.org/standard